

LAPORAN PRAKTIKUM STRUKTUR DATA

IMPLEMENTASI SORTING dan GUI PADA JAVA

Disusun Oleh:

Ihsanul Zaky EL-Muhammady

NIM:2511533001

Dosen Pengampu:

Dr.Wahyudi S.T , M.T

Asisten Laboratorium:

Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Dengan penuh rasa syukur, saya mengucapkan terima kasih kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum "Implementasi Sorting dan GUI pada Java" ini dapat diselesaikan dengan baik.

Saya juga menyampaikan penghargaan kepada dosen pengampu Mata Kuliah Praktikum Struktur Data serta para asisten laboratorium yang telah memberikan arahan dan bimbingan selama proses praktikum berlangsung.

Laporan ini disusun sebagai dokumentasi kegiatan praktikum yang bertujuan untuk memahami algoritma sorting seperti Shell Sort, Merge Sort, dan Bubble Sort, serta implementasinya dalam antarmuka grafis (GUI) menggunakan Java Swing, mulai dari perancangan komponen dasar hingga integrasi algoritma ke dalam aplikasi secara visual.

Saya menyadari laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan agar laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai implementasi sorting dan GUI pada Java.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	
DAFTAR ISI.....	
BAB I PENDAHULUAN	
1.1 Latar Belakang	
1.2 Tujuan	
1.3 Manfaat.....	
BAB II PEMBAHASAN	
2.1 ShellSort_2511533001	
2.2 QuickSort_2511533001	
2.3 MergeSort_2511533001	
2.4 BubbleSortGUI_2511533001	
2.5 MergeSortGUI_2511533001	
2.6 Tugas	
BAB III KESIMPULAN	
3.1 Kesimpulan	
3.2 Saran.....	
DAFTAR PUSTAKA.....	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, proses pengurutan (sorting) merupakan salah satu teknik yang paling sering digunakan untuk mempermudah pencarian, pengelompokan, dan penyajian informasi. Data yang tersusun dengan rapi akan lebih mudah diproses serta memberikan hasil yang lebih efektif dibandingkan data yang masih acak.

Terdapat berbagai algoritma sorting yang dapat digunakan, di antaranya Bubble Sort, Shell Sort, dan Merge Sort. Bubble Sort merupakan algoritma sederhana yang bekerja dengan membandingkan elemen yang berdekatan dan menukarnya jika tidak sesuai urutan. Shell Sort merupakan pengembangan dari Insertion Sort yang menggunakan jarak tertentu (gap) untuk mempercepat proses pengurutan. Sementara itu, Merge Sort menggunakan metode divide and conquer dengan membagi data menjadi beberapa bagian, kemudian menggabungkannya kembali dalam keadaan terurut.

Selain memahami algoritma sorting, penyajian hasil pengurutan juga menjadi aspek penting dalam pengembangan aplikasi. Oleh karena itu, digunakan antarmuka grafis (GUI) berbasis Java Swing melalui program BubbleSortGUI dan MergeSortGUI agar proses pengurutan dapat ditampilkan secara lebih interaktif dan mudah dipahami oleh pengguna. Melalui praktikum ini, mahasiswa dapat mempelajari implementasi algoritma sorting sekaligus penerapannya dalam aplikasi berbasis GUI.

1.2 Tujuan

1. Memahami konsep dan cara kerja algoritma Bubble Sort, Shell Sort, dan Merge **Sort**.
2. Mengetahui perbedaan karakteristik serta efisiensi dari masing-masing algoritma sorting.
3. Mempelajari implementasi algoritma sorting dalam bahasa pemrograman Java.
4. Membangun aplikasi GUI menggunakan Java Swing melalui program BubbleSortGUI dan MergeSortGUI.
5. Melatih kemampuan dalam menggabungkan logika algoritma dengan antarmuka grafis yang interaktif.

1.3 Manfaat

1. Meningkatkan pemahaman mengenai algoritma Bubble Sort, Shell Sort, dan Merge Sort.
2. Menambah keterampilan dalam mengimplementasikan algoritma sorting menggunakan Java.

3. Memberikan pengalaman dalam pembuatan aplikasi berbasis GUI menggunakan Java Swing.
4. Memahami cara menampilkan hasil pengurutan data secara visual melalui BubbleSortGUI dan MergeSortGUI.
5. Mengembangkan kemampuan pemrograman dalam merancang aplikasi yang terstruktur, efisien, dan mudah digunakan.

BAB II

PEMBAHASAN

2.1 ShellSort_2511533001

```
1 package pekan8_2511533001;
2
3 public class ShellSort_2511533001 {
4
5     public static void ShellSort_2511533001(int[] A) {
6         int n_3001 = A.length;
7         int gap_3001 = n_3001 / 2;
8
9         while (gap_3001 > 0) {
10             for (int i_3001 = gap_3001; i_3001 < n_3001; i_3001++) {
11                 int temp_3001 = A[i_3001];
12                 int j_3001 = i_3001;
13
14                 while (j_3001 >= gap_3001 && A[j_3001 - gap_3001] > temp_3001) {
15                     A[j_3001] = A[j_3001 - gap_3001];
16                     j_3001 = j_3001 - gap_3001;
17                 }
18
19                 A[j_3001] = temp_3001;
20             }
21             gap_3001 = gap_3001 / 2;
22         }
23     }
24
25     public static void main(String[] args) {
26         int[] data_3001 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
27
28         System.out.print("Sebelum : ");
29         printArray_2511533001(data_3001);
30
31         ShellSort_2511533001(data_3001);
32
33         System.out.print("Sesudah (Shell Sort): ");
34         printArray_2511533001(data_3001);
35
36     }
37
38     public static void printArray_2511533001(int[] arr) {
39         for (int i_3001 : arr)
40             System.out.print(i_3001 + " ");
41         System.out.println();
42     }
43 }
```

Penjelasan:

Class ini mengimplementasikan algoritma Shell Sort, yaitu algoritma pengurutan yang merupakan pengembangan dari Insertion Sort. Shell Sort bekerja dengan membandingkan dan mengurutkan elemen-elemen yang memiliki jarak tertentu (gap), sehingga data dapat lebih cepat mendekati kondisi terurut sebelum dilakukan pengurutan akhir.

Di dalam method ShellSort_2511533001, variabel gap_3001 digunakan untuk menentukan jarak antar elemen yang akan dibandingkan. Nilai gap awal ditentukan sebesar setengah dari panjang array, kemudian akan terus dibagi dua hingga bernilai 0. Pada setiap nilai gap, proses pengurutan dilakukan dengan konsep Insertion Sort. Elemen yang lebih besar akan digeser ke belakang hingga ditemukan posisi yang tepat untuk menyisipkan elemen yang sedang diproses. Setelah seluruh proses selesai, array akan tersusun secara berurutan dari nilai terkecil ke terbesar.

Output:

```
Sebelum : 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.2 QuickSort_2511533001

```
1 package pekan8_2511533001;
2
3 public class QuickSort_2511533001 {
4
5     static void swap_3001(int[] arr_3001, int i_3001, int j_3001) {
6         int temp_3001 = arr_3001[i_3001];
7         arr_3001[i_3001] = arr_3001[j_3001];
8         arr_3001[j_3001] = temp_3001;
9     }
10
11     // metode tambahan untuk mengatur pivot menggunakan median of three
12     static void medianOfThree_3001(int[] arr_3001, int low_3001, int high_3001) {
13         int mid_3001 = low_3001 + (high_3001 - low_3001) / 2;
14
15         // urutkan elemen low, mid, high
16         if (arr_3001[low_3001] > arr_3001[mid_3001]) {
17             swap_3001(arr_3001, low_3001, mid_3001);
18         }
19
20         if (arr_3001[low_3001] > arr_3001[high_3001]) {
21             swap_3001(arr_3001, low_3001, high_3001);
22         }
23
24         if (arr_3001[mid_3001] > arr_3001[high_3001]) {
25             swap_3001(arr_3001, mid_3001, high_3001);
26         }
27
28         swap_3001(arr_3001, mid_3001, high_3001);
29     }
30
31     static int partition_3001(int[] arr_3001, int low_3001, int high_3001) {
32         // panggil fungsi medianOfThree sebelum menentukan pivot
33         medianOfThree_3001(arr_3001, low_3001, high_3001);
34
35         int pivot_3001 = arr_3001[high_3001];
36
37         int i_3001 = (low_3001 - 1);
38
39         for (int j_3001 = low_3001; j_3001 <= high_3001 - 1; j_3001++) {
40             // jika elemen saat ini lebih kecil dari pivot
41             if (arr_3001[j_3001] < pivot_3001) {
42                 i_3001++;
43                 swap_3001(arr_3001, i_3001, j_3001);
44             }
45         }
46
47         swap_3001(arr_3001, i_3001 + 1, high_3001);
48         return (i_3001 + 1);
49     }
50
51     static void quickSort_2511533001(int[] arr_3001, int low_3001, int high_3001) {
52         if (low_3001 < high_3001) {
53             int pi_3001 = partition_3001(arr_3001, low_3001, high_3001);
54
55             quickSort_2511533001(arr_3001, low_3001, pi_3001 - 1);
56             quickSort_2511533001(arr_3001, pi_3001 + 1, high_3001);
57         }
58     }
59
60     public static void printArr_3001(int[] arr_3001) {
61         for (int i_3001 = 0; i_3001 < arr_3001.length; i_3001++) {
62             System.out.print(arr_3001[i_3001] + " ");
63         }
64         System.out.println();
65     }
66
67     public static void main(String[] args) {
68         int arr_3001[] = {10, 7, 8, 9, 1, 5};
69         int n_3001 = arr_3001.length;
70
71         System.out.print("Data sebelum diurutkan: ");
72         printArr_3001(arr_3001);
73
74         quickSort_2511533001(arr_3001, 0, n_3001 - 1);
75
76         System.out.print("Data Terurut quicksort : ");
77         printArr_3001(arr_3001);
78     }
```

Penjelasan:

Class ini mengimplementasikan algoritma Quick Sort, yaitu algoritma pengurutan yang menggunakan metode divide and conquer dengan memilih sebuah elemen sebagai pivot untuk membagi data menjadi dua bagian. Data yang lebih kecil dari pivot ditempatkan di sebelah kiri, sedangkan data yang lebih besar ditempatkan di sebelah kanan.

Pada program ini terdapat method medianOfThree_3001 yang digunakan untuk memilih pivot menggunakan teknik median of three. Teknik ini membandingkan elemen pertama, tengah, dan terakhir untuk memperoleh pivot yang lebih baik sehingga dapat mengurangi kemungkinan terjadinya pembagian data yang tidak seimbang. Selanjutnya method partition_3001 bertugas memisahkan elemen-elemen berdasarkan nilai pivot tersebut. Setelah proses partisi selesai, method quickSort_2511533001 akan memanggil dirinya sendiri secara rekursif untuk mengurutkan bagian kiri dan kanan hingga seluruh data berada dalam urutan yang benar.

Output:

```
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort : 1 5 7 8 9 10
```

2.3 MergeSort_2511533001

```
package pekan8_2511533001;

public class MergeSort_2511533001 {

    void merge_3001(int[] arr_3001, int l_3001, int m_3001, int r_3001) {
        // find sizes of two subarrays to be merged
        int n1_3001 = m_3001 - l_3001 + 1;
        int n2_3001 = r_3001 - m_3001;

        /* create temp arrays */
        int L_3001[] = new int[n1_3001];
        int R_3001[] = new int[n2_3001];

        /* copy data to temp arrays */
        for (int i_3001 = 0; i_3001 < n1_3001; ++i_3001)
            L_3001[i_3001] = arr_3001[l_3001 + i_3001];

        for (int j_3001 = 0; j_3001 < n2_3001; ++j_3001)
            R_3001[j_3001] = arr_3001[m_3001 + 1 + j_3001];

        int i_3001 = 0, j_3001 = 0;

        // initial index merged subarray array
        int k_3001 = l_3001;

        while (i_3001 < n1_3001 && j_3001 < n2_3001) {
            if (L_3001[i_3001] <= R_3001[j_3001]) {
                arr_3001[k_3001] = L_3001[i_3001];
                i_3001++;
            } else {
                arr_3001[k_3001] = R_3001[j_3001];
                j_3001++;
            }
            k_3001++;
        }

        while (i_3001 < n1_3001) {
            arr_3001[k_3001] = L_3001[i_3001];
            i_3001++;
            k_3001++;
        }

        /* copy remaining elements of R[] if any */
        while (j_3001 < n2_3001) {
            arr_3001[k_3001] = R_3001[j_3001];
            j_3001++;
            k_3001++;
        }
    }

    void sort_3001(int arr_3001[], int l_3001, int r_3001) {
        if (l_3001 < r_3001) {
            // find the middle point
            int m_3001 = (l_3001 + r_3001) / 2;

            // sort first and second halves
            sort_3001(arr_3001, l_3001, m_3001);
            sort_3001(arr_3001, m_3001 + 1, r_3001);

            // merge the sorted halves
            merge_3001(arr_3001, l_3001, m_3001, r_3001);
        }
    }

    /* a utility function to print array of size n */
    static void printArray_3001(int arr_3001[]) {
        int n_3001 = arr_3001.length;

        for (int i_3001 = 0; i_3001 < n_3001; ++i_3001)
            System.out.print(arr_3001[i_3001] + " ");
    }
}
```



```

    }
    System.out.println("/");
}

public static void main(String args[]) {
    int arr_3001[] = {12, 11, 13, 5, 6, 7};

    System.out.println("Sebelum terurut: ");
    printArray_3001(arr_3001);

    MergeSort_2511533001 ob_3001 = new MergeSort_2511533001();
    ob_3001.sort_3001(arr_3001, 0, arr_3001.length - 1);

    System.out.println("\nSesudah Terurut menggunakan merge sort");
    printArray_3001(arr_3001);
}
}

```

Penjelasan:

Class ini mengimplementasikan algoritma Merge Sort, yaitu algoritma pengurutan yang menggunakan pendekatan divide and conquer. Cara kerjanya adalah dengan membagi array menjadi beberapa bagian yang lebih kecil, kemudian mengurutkannya dan menggabungkannya kembali menjadi satu array yang terurut.

Di dalam method sort_3001, array akan terus dibagi menjadi dua bagian menggunakan indeks tengah hingga setiap bagian hanya berisi satu elemen. Setelah itu method merge_3001 digunakan untuk menggabungkan kembali bagian-bagian tersebut. Pada proses penggabungan, dua array sementara yaitu L_3001 dan R_3001 digunakan untuk menyimpan data dari sisi kiri dan kanan. Elemen-elemen dari kedua array tersebut kemudian dibandingkan satu per satu, lalu disalin kembali ke array utama dalam urutan yang benar. Proses ini berlangsung hingga seluruh data berhasil diurutkan.

Output:

```

Sebelum terurut:
12 11 13 5 6 7

Sesudah Terurut menggunakan merge sort
5 6 7 11 12 13

```

2.4 BubbleSortGUI_2511533001

```

private void setArrayFromInput_3001() {
    String text_3001 = inputField_3001.getText().trim();
    if (text_3001.isEmpty()) return;
    String[] parts_3001 = text_3001.split(",");
    array_3001 = new int[parts_3001.length];
    try {
        for (int k_3001 = 0; k_3001 < parts_3001.length; k_3001++) {
            array_3001[k_3001] = Integer.parseInt(parts_3001[k_3001].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukan hanya angka dipisahkan dengan koma!", "Error",
            JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_3001 = 0;
    j_3001 = 0;
    stepCount_3001 = 1;
    sorting_3001 = true;
    stepButton_3001.setEnabled(true);
    stepArea_3001.setText("");
    panelArray_3001.removeAll();
    labelArray_3001 = new JLabel[array_3001.length];
    for (int k_3001 = 0; k_3001 < array_3001.length; k_3001++) {
        labelArray_3001[k_3001] = new JLabel(String.valueOf(array_3001[k_3001]));
        labelArray_3001[k_3001].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_3001[k_3001].setOpaque(true);
        labelArray_3001[k_3001].setBackground(Color.WHITE);
        labelArray_3001[k_3001].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_3001[k_3001].setPreferredSize(new Dimension(50, 50));
        labelArray_3001[k_3001].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_3001.add(labelArray_3001[k_3001]);
    }
    panelArray_3001.revalidate();
}

```

```

private void performStep_3001() {
    if (!sorting_3001 || i_3001 >= array_3001.length - 1) {
        sorting_3001 = false;
        stepButton_3001.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }

    resetHighlights_3001();
    StringBuilder stepLog_3001 = new StringBuilder();

    labelArray_3001[j_3001].setBackground(Color.CYAN);
    labelArray_3001[j_3001 + 1].setBackground(Color.CYAN);

    if (array_3001[j_3001] > array_3001[j_3001 + 1]) {
        // swap
        int temp_3001 = array_3001[j_3001];
        array_3001[j_3001] = array_3001[j_3001 + 1];
        array_3001[j_3001 + 1] = temp_3001;

        labelArray_3001[j_3001].setBackground(Color.RED);
        labelArray_3001[j_3001 + 1].setBackground(Color.RED);

        stepLog_3001.append("Langkah ")
            .append(stepCount_3001)
            .append(":Menukar elemen ke-")
            .append(j_3001)
            .append(" ")
            .append(array_3001[j_3001 + 1])
            .append(" dengan ke-")
            .append(j_3001 + 1)
            .append(" ")
            .append(array_3001[j_3001])
            .append("\n");
    }
    else {
        stepLog_3001.append("Langkah ")
            .append(stepCount_3001)
            .append(":Tidak ada pertukaran elemen ke-")
            .append(j_3001)
            .append(" dan ke-")
            .append(j_3001 + 1)
            .append("\n");
    }

    stepLog_3001.append("Hasil : ")
        .append(arrayToString_3001(array_3001))
        .append("\n\n");

    stepArea_3001.append(stepLog_3001.toString());

    updateLabels_3001();

    j_3001++;

    if (j_3001 >= array_3001.length - i_3001 - 1) {
        j_3001 = 0;
        i_3001++;
    }

    stepCount_3001++;

    if (i_3001 >= array_3001.length - 1) {
        sorting_3001 = false;
        stepButton_3001.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void undateLabels_3001() {

```

```

private void updateLabels_3001() {
    for (int k_3001 = 0; k_3001 < array_3001.length; k_3001++) {
        labelArray_3001[k_3001].setText(String.valueOf(array_3001[k_3001]));
    }
}

private void resetHighlights_3001() {
    for (JLabel label_3001 : labelArray_3001) {
        label_3001.setBackground(Color.WHITE);
    }
}

private void reset_3001() {
    inputField_3001.setText("");
    panelArray_3001.removeAll();
    panelArray_3001.revalidate();
    panelArray_3001.repaint();
    stepArea_3001.setText("");
    stepButton_3001.setEnabled(false);
    sorting_3001 = false;
    i_3001 = 0;
    j_3001 = 0;
    stepCount_3001 = 1;
}

private String arrayToString_3001(int[] arr_3001) {
    StringBuilder sb_3001 = new StringBuilder();

    for (int k_3001 = 0; k_3001 < arr_3001.length; k_3001++) {
        sb_3001.append(arr_3001[k_3001]);

        if (k_3001 < arr_3001.length - 1)
            sb_3001.append(", ");
    }

    return sb_3001.toString();
}
// FIX 4: Tambah method main agar bisa dijalankan
public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                BubbleSortGUI_2511533001 frame = new BubbleSortGUI_2511533001();
                frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}

```

Penjelasan:

Class ini merupakan implementasi algoritma Bubble Sort dalam bentuk aplikasi GUI menggunakan Java Swing. Program dibuat agar pengguna dapat melihat proses pengurutan data secara bertahap sehingga cara kerja Bubble Sort menjadi lebih mudah dipahami dibandingkan hanya melihat hasil akhirnya.

Pada bagian awal, pengguna dapat memasukkan sejumlah angka melalui inputField_3001 dengan format dipisahkan menggunakan koma. Ketika tombol Set Array ditekan, method setArrayFromInput_3001() akan membaca input, mengubahnya menjadi array integer, lalu menampilkan setiap elemen dalam bentuk kotak-kotak pada panel visual. Selain itu, variabel-variabel kontrol seperti i_3001, j_3001, dan stepCount_3001 akan diinisialisasi untuk memulai proses pengurutan.

Proses utama Bubble Sort dijalankan pada method performStep_3001(). Setiap kali tombol Langkah Selanjutnya ditekan, program akan membandingkan dua elemen yang berdekatan menggunakan indeks j_3001 dan j_3001 + 1. Jika elemen kiri lebih besar daripada elemen

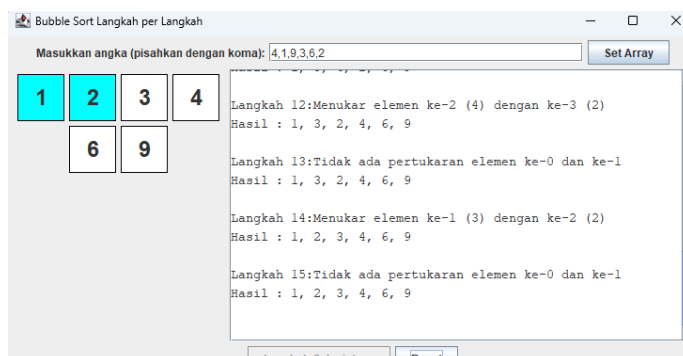
kanan, maka dilakukan pertukaran posisi (swap). Setelah itu tampilan array diperbarui sehingga pengguna dapat melihat perubahan yang terjadi secara langsung.

Untuk membantu visualisasi, elemen yang sedang dibandingkan akan diberi warna biru (cyan). Jika terjadi pertukaran data, kedua elemen tersebut akan berubah menjadi merah, menandakan bahwa proses swap sedang dilakukan. Setiap langkah yang terjadi juga dicatat pada stepArea_3001, sehingga pengguna dapat melihat urutan proses pengurutan beserta hasil array setelah setiap perbandingan.

Method updateLabels_3001() digunakan untuk memperbarui nilai yang ditampilkan pada setiap kotak setelah terjadi perubahan data. Sedangkan method resetHighlights_3001() berfungsi mengembalikan warna seluruh elemen ke kondisi semula sebelum langkah berikutnya dijalankan.

Selain itu, program juga menyediakan method reset_3001() yang digunakan untuk menghapus data, membersihkan tampilan, serta mengembalikan seluruh variabel ke kondisi awal sehingga pengguna dapat melakukan percobaan pengurutan baru. Setelah seluruh elemen berhasil diurutkan, program akan menampilkan pesan bahwa proses Bubble Sort telah selesai dan tombol langkah berikutnya akan dinonaktifkan.

Output:



2.5 MergeSortGUI 2511533001

```
private void SetMergeInput_3001() {
    String text_3001 = inputField_3001.getText().trim();
    if (text_3001.isEmpty()) return;

    String[] parts_3001 = text_3001.split(",");
    array_3001 = new int[parts_3001.length];

    try {
        for (int i_3001 = 0; i_3001 < parts_3001.length; i_3001++) {
            array_3001[i_3001] = Integer.parseInt(parts_3001[i_3001].trim());
        }
    } catch (NumberFormatException e_3001) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
            "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    labelArray_3001 = new JLabel[array_3001.length];
    panelArray_3001.removeAll();

    for (int i_3001 = 0; i_3001 < array_3001.length; i_3001++) {
        labelArray_3001[i_3001] = new JLabel(String.valueOf(array_3001[i_3001]));
        labelArray_3001[i_3001].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_3001[i_3001].setOpaque(true);
        labelArray_3001[i_3001].setBackground(Color.WHITE);
        labelArray_3001[i_3001].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_3001[i_3001].setPreferredSize(new Dimension(50, 50));
        labelArray_3001[i_3001].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_3001.add(labelArray_3001[i_3001]);
    }

    mergeQueue_3001.clear();
    generateMergeSteps_3001(0, array_3001.length - 1);
}
```

```

        stepButton_3001.setEnabled(true);
        stepArea_3001.setText("");
        stepCount_3001 = 1;
        isMerging_3001 = false;
        copying_3001 = false;

        panelArray_3001.revalidate();
        panelArray_3001.repaint();
    }

    private void generateMergeSteps_3001(int left_3001, int right_3001) {
        if (left_3001 < right_3001) {
            int mid_3001 = left_3001 + (right_3001 - left_3001) / 2;

            generateMergeSteps_3001(left_3001, mid_3001);
            generateMergeSteps_3001(mid_3001 + 1, right_3001);

            mergeQueue_3001.add(new int[] { left_3001, mid_3001, right_3001 });
        }
    }

    private void performStep_3001() {
        resetHighlights_3001();

        if (!isMerging_3001 && !mergeQueue_3001.isEmpty()) {
            int[] range_3001 = mergeQueue_3001.poll();

            left_3001 = range_3001[0];
            mid_3001 = range_3001[1];
            right_3001 = range_3001[2];

            temp_3001 = new int[right_3001 - left_3001 + 1];
            i_3001 = left_3001;
            j_3001 = mid_3001 + 1;
            k_3001 = 0;

            copying_3001 = false;
            isMerging_3001 = true;

            stepArea_3001.append(
                "Langkah " + stepCount_3001++
                + ": Mulai merge dari "
                + left_3001 + " ke "
                + right_3001 + "\n"
            );

            return;
        }

        if (isMerging_3001 && !copying_3001) {
            if (i_3001 <= mid_3001 && j_3001 <= right_3001) {
                labelArray_3001[i_3001].setBackground(Color.CYAN);
                labelArray_3001[j_3001].setBackground(Color.CYAN);

                if (array_3001[i_3001] <= array_3001[j_3001]) {
                    temp_3001[k_3001++] = array_3001[i_3001++];
                } else {
                    temp_3001[k_3001++] = array_3001[j_3001++];
                }

                stepArea_3001.append(
                    "Langkah " + stepCount_3001++
                    + ": Bandingkan dan salin elemen\n"
                );

                return;
            } else if (i_3001 <= mid_3001) {
                temp_3001[k_3001++] = array_3001[i_3001++];

                stepArea_3001.append(
                    "Langkah " + stepCount_3001++
                    + ": Salin sisa kiri\n"
                );

                return;
            } else if (j_3001 <= right_3001) {
                temp_3001[k_3001++] = array_3001[j_3001++];

                stepArea_3001.append(
                    "Langkah " + stepCount_3001++
                    + ": Salin sisa kanan\n"
                );

                return;
            } else {
                copying_3001 = true;
                k_3001 = 0;
                return;
            }
        }

        if (copying_3001 && k_3001 < temp_3001.length) {
            array_3001[left_3001 + k_3001] = temp_3001[k_3001];
            labelArray_3001[left_3001 + k_3001].setText(String.valueOf(temp_3001[k_3001]));
            labelArray_3001[left_3001 + k_3001].setBackground(Color.GREEN);
        }
    }

```

```

        k_3001++;

        stepArea_3001.append("Langkah " + stepCount_3001++ + ": Tempelkan ke array utama\n");
        return;
    }

    if (copying_3001 && k_3001 == temp_3001.length) {
        isMerging_3001 = false;
        copying_3001 = false;
    }

    if (mergeQueue_3001.isEmpty() && !isMerging_3001) {
        stepArea_3001.append("Selesai.\n");
        stepButton_3001.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
    }
}

private void resetHighlights_3001() {
    if (labelArray_3001 == null) return;

    for (JLabel label_3001 : labelArray_3001) {
        label_3001.setBackground(Color.WHITE);
    }
}

private void reset_3001() {
    inputField_3001.setText("");
    panelArray_3001.removeAll();
    panelArray_3001.revalidate();
    panelArray_3001.repaint();
    stepArea_3001.setText("");
    stepButton_3001.setEnabled(false);
}

mergeQueue_3001.clear();
isMerging_3001 = false;
copying_3001 = false;
stepCount_3001 = 1;
}

public static void main(String[] args_3001) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511533001 frame_3001 = new MergeSortGUI_2511533001();
        frame_3001.setVisible(true);
    });
}
}

```

Penjelasan:

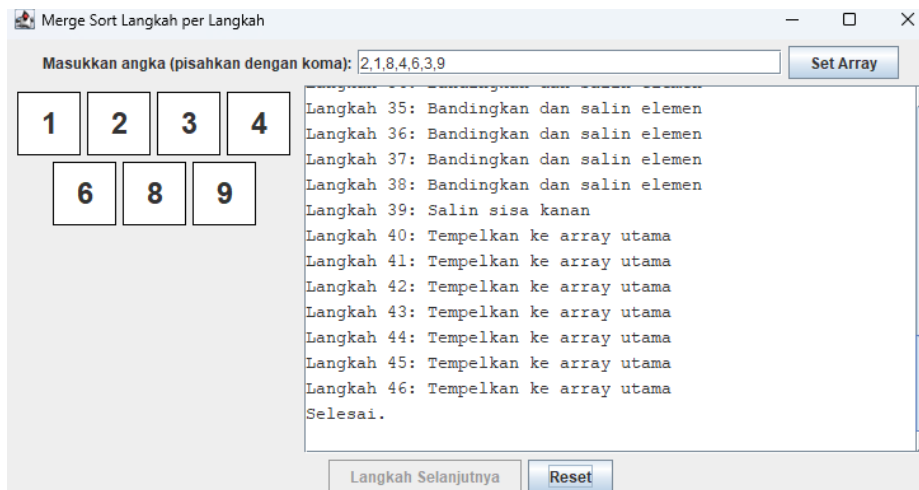
Class ini merupakan implementasi algoritma Merge Sort dalam bentuk aplikasi GUI menggunakan Java Swing. Program dirancang agar pengguna dapat melihat proses pengurutan secara bertahap sehingga lebih mudah memahami cara kerja Merge Sort.

Pengguna dapat memasukkan data melalui inputField_3001, kemudian data tersebut akan ditampilkan dalam bentuk kotak-kotak pada panel visual. Method generateMergeSteps_3001 bertugas menyimpan seluruh tahapan pembagian dan penggabungan array ke dalam mergeQueue_3001. Selanjutnya, setiap kali tombol Langkah Selanjutnya ditekan, method performStep_3001 akan menjalankan satu langkah proses Merge Sort.

Selama proses berlangsung, elemen yang sedang dibandingkan akan diberi warna biru (*cyan*), sedangkan elemen yang sudah ditempatkan kembali ke array utama akan diberi warna hijau. Semua aktivitas yang terjadi juga dicatat pada stepArea_3001 sehingga pengguna dapat mengikuti urutan langkah pengurutan secara detail. Setelah seluruh proses merge selesai,

program akan menampilkan pesan bahwa pengurutan telah berhasil dilakukan dan data telah tersusun secara terurut.

Output:



2.6 Tugas

1.Lagu_2511533001

```
package pekan8_2511533001;

public class Lagu_2511533001 {
    String judul_3001;
    String penyanyi_3001;
    int durasi_3001;

    public Lagu_2511533001(String judul, String penyanyi, int durasi) {
        this.judul_3001 = judul;
        this.penyanyi_3001 = penyanyi;
        this.durasi_3001 = durasi;
    }
}
```

Penjelasan:

Class Lagu_2511533001 merupakan class yang digunakan untuk merepresentasikan data sebuah lagu. Class ini memiliki tiga atribut utama, yaitu judul_3001 untuk menyimpan judul lagu, penyanyi_3001 untuk menyimpan nama penyanyi, dan durasi_3001 untuk menyimpan lama durasi lagu dalam satuan detik.

Di dalam class ini terdapat constructor Lagu_2511533001 yang digunakan untuk menginisialisasi nilai dari setiap atribut saat objek lagu dibuat. Ketika sebuah objek dibuat, nilai judul, penyanyi, dan durasi yang diberikan sebagai parameter akan disimpan ke dalam atribut class menggunakan keyword this.

Class ini berfungsi sebagai struktur data yang menyimpan informasi setiap lagu dan menjadi objek yang akan diproses pada class Sorting_2511533001, baik saat penyimpanan data maupun saat proses pengurutan berlangsung.

2.Sorting_2511533001

```
package pekan8_2511533001;

public class Sorting_2511533001 {

    static Lagu_2511533001[] dataLagu_3001 = new Lagu_2511533001[20];
    static int jumlahData_3001 = 0;

    public static void inputData_3001() {

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Monokrom", "Tulus", 214);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Hati-Hati di Jalan", "Tulus", 242);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Rumah ke Rumah", "Hindia", 276);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Evaluasi", "Hindia", 234);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Zona Nyaman", "Fourtwnty", 230);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Akad", "Payung Teduh", 269);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Sampai Jadi Debu", "Banda Neira", 298);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Bertaut", "Nadin Amizah", 315);

        dataLagu_3001[jumlahData_3001++] =
            new Lagu_2511533001("Untuk Perempuan yang Sedang Dalam Pelukan", "Payung Teduh", 337);

    }

    public static void insertionSort_3001() {

        for (int i_3001 = 1; i_3001 < jumlahData_3001; i_3001++) {

            Lagu_2511533001 kunci_3001 = dataLagu_3001[i_3001];
            int j_3001 = i_3001 - 1;

            while (j_3001 >= 0 &&
                dataLagu_3001[j_3001].durasi_3001 > kunci_3001.durasi_3001) {

                dataLagu_3001[j_3001 + 1] = dataLagu_3001[j_3001];
                j_3001--;

            }

            dataLagu_3001[j_3001 + 1] = kunci_3001;

        }

    }

    public static void tampilData_3001() {

        for (int i_3001 = 0; i_3001 < jumlahData_3001; i_3001++) {

            System.out.println(
                (i_3001 + 1) + ". "
                + dataLagu_3001[i_3001].judul_3001 + " | "
                + dataLagu_3001[i_3001].penyanyi_3001 + " | "
                + dataLagu_3001[i_3001].durasi_3001 + " detik");

        }

    }

    public static void main(String[] args) {

        inputData_3001();

        System.out.println("=== DATA SEBELUM SORTING ===");

        tampilData_3001();

        insertionSort_3001();

        System.out.println("\n=== DATA SETELAH SORTING DURASI ===");
        tampilData_3001();

    }

}
```

Penjelasan:

Class ini mengimplementasikan algoritma Insertion Sort, yaitu algoritma pengurutan yang bekerja dengan cara menyisipkan setiap elemen ke posisi yang sesuai pada bagian data yang

sudah terurut. Pada program ini, data yang diurutkan berupa kumpulan objek lagu yang berisi judul lagu, nama penyanyi, dan durasi lagu.

Di dalam method `inputData_3001`, beberapa objek lagu dibuat dan disimpan ke dalam array `dataLagu_3001`. Setiap objek berisi informasi judul lagu, penyanyi, dan durasi yang nantinya akan digunakan sebagai data yang akan diurutkan.

Proses pengurutan dilakukan pada method `insertionSort_3001`. Method ini mengambil satu data lagu sebagai kunci (`kunci_3001`), kemudian membandingkan durasinya dengan data-data yang berada di sebelah kiri. Jika ditemukan durasi yang lebih besar, maka data tersebut akan digeser satu posisi ke kanan. Proses ini terus dilakukan sampai ditemukan posisi yang sesuai untuk menyisipkan data kunci. Dengan cara tersebut, data lagu akan tersusun secara bertahap berdasarkan durasi dari yang paling pendek hingga yang paling panjang.

Setelah proses pengurutan selesai, method `tampilData_3001` digunakan untuk menampilkan seluruh data lagu ke layar. Method ini melakukan perulangan pada array dan mencetak informasi setiap lagu berupa judul, penyanyi, serta durasinya. Hasil akhirnya adalah daftar lagu yang telah terurut berdasarkan durasi secara ascending (kecil ke besar).

Output:

```
=== DATA SEBELUM SORTING ===
1. Monokrom | Tulus | 214 detik
2. Hati-Hati di Jalan | Tulus | 242 detik
3. Rumah ke Rumah | Hindia | 276 detik
4. Evaluasi | Hindia | 234 detik
5. Zona Nyaman | Fourtwnty | 230 detik
6. Akad | Payung Teduh | 269 detik
7. Sampai Jadi Debu | Banda Neira | 298 detik
8. Bertaut | Nadin Amizah | 315 detik
9. Untuk Perempuan yang Sedang Dalam Pelukan | Payung Teduh | 337 detik
10. Kuning | Rumahsakit | 258 detik

=== DATA SETELAH SORTING DURASI ===
1. Monokrom | Tulus | 214 detik
2. Zona Nyaman | Fourtwnty | 230 detik
3. Evaluasi | Hindia | 234 detik
4. Hati-Hati di Jalan | Tulus | 242 detik
5. Kuning | Rumahsakit | 258 detik
6. Akad | Payung Teduh | 269 detik
7. Rumah ke Rumah | Hindia | 276 detik
8. Sampai Jadi Debu | Banda Neira | 298 detik
9. Bertaut | Nadin Amizah | 315 detik
10. Untuk Perempuan yang Sedang Dalam Pelukan | Payung Teduh | 337 detik
```

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum mengenai implementasi berbagai algoritma sorting pada Java, dapat disimpulkan bahwa algoritma sorting merupakan metode yang digunakan untuk mengurutkan data sehingga menjadi lebih terstruktur dan mudah diproses. Pada praktikum ini diterapkan beberapa algoritma pengurutan, yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort yang masing-masing memiliki cara kerja dan tingkat efisiensi yang berbeda dalam mengurutkan data.

Bubble Sort melakukan pengurutan dengan cara membandingkan elemen yang berdekatan dan menukarnya apabila posisinya tidak sesuai. Shell Sort mengembangkan konsep Insertion Sort dengan menggunakan jarak tertentu (gap) sehingga proses pengurutan menjadi lebih cepat. Quick Sort menggunakan metode divide and conquer dengan memilih sebuah pivot untuk membagi data menjadi beberapa bagian sebelum diurutkan kembali. Sementara itu, Merge Sort juga menggunakan pendekatan divide and conquer dengan membagi data menjadi bagian-bagian yang lebih kecil, kemudian menggabungkannya kembali dalam keadaan terurut.

Selain mempelajari algoritma sorting, praktikum ini juga menerapkan Graphical User Interface (GUI) menggunakan Java Swing melalui program BubbleSortGUI dan MergeSortGUI. Penggunaan GUI memungkinkan proses pengurutan data ditampilkan secara visual sehingga pengguna dapat melihat setiap langkah yang dilakukan algoritma sorting secara lebih jelas dan interaktif. Komponen seperti JFrame, JPanel, JButton, JTextField, JLabel, dan JTextArea digunakan untuk membangun antarmuka program dan menampilkan proses pengurutan secara langsung.

Penerapan algoritma sorting dalam bentuk GUI membantu pengguna memahami alur kerja setiap algoritma secara lebih mudah dibandingkan hanya melihat hasil akhir pengurutan. Visualisasi tersebut menunjukkan bagaimana data dibandingkan, dipindahkan, maupun digabungkan hingga menghasilkan data yang telah terurut dengan benar.

Dengan demikian, praktikum implementasi algoritma sorting dan GUI pada Java tidak hanya memberikan pemahaman mengenai konsep dan cara kerja berbagai metode pengurutan data, tetapi juga meningkatkan kemampuan dalam menerapkan pemrograman berorientasi objek, penggunaan Java Swing, pengelolaan array, serta pengembangan aplikasi sederhana yang bersifat interaktif.

3.2 Saran

Sebaiknya dosen menyelenggarakan sesi pra-praktikum di kelas sebagai pengantar materi, agar mahasiswa memperoleh pemahaman awal yang lebih baik. Dengan demikian, potensi kepanikan atau kesalahan selama praktikum dapat diminimalkan. Disarankan agar materi praktikum dibagikan terlebih dahulu melalui platform iLearn.

DAFTAR PUSTAKA

[1] Oracle Corporation, “The Java™ Tutorials,” Oracle Documentation. [Online].

Available: <https://docs.oracle.com/javase/tutorial/>

[2] Oracle Corporation, “Creating a GUI With Swing,” Oracle Documentation. [Online].

Available: <https://docs.oracle.com/javase/tutorial/uiswing/>

[3] Oracle Corporation, “The Collections Framework,” Oracle Documentation. [Online].

Available: <https://docs.oracle.com/javase/tutorial/collections/>